Reinforcement Learning

EQUATION DE BELLMAN et la PROGRAMMATION DYNAMIQUE

Etude des algorithmes d'apprentissage pour trouver la politique optimale.

Base : Equation de Bellman pour l'apprentissage par renforcement
Utilisée pour trouver la valeur optimale et les Q functions de façon récursives.

Equation de Bellman (du nom de Richard Bellman) est une aide à résoudre les MDP. C-AD trouver la politique optimale.

L'équation de Bellman stipule que la valeur d'un état peut être obtenue par la somme du reward immédiat et de la valeur actualisée de l'état suivant.

Reinforcement Learning

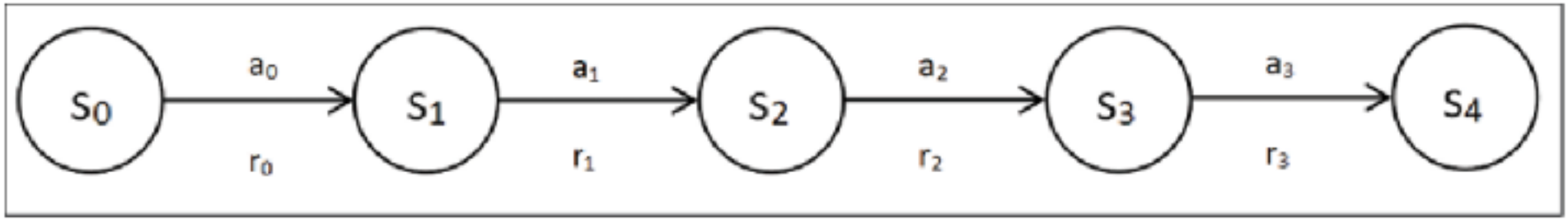
Supposons que nous effectuions une action a dans l'état s et passons à l'état suivant s' et obtenant un reward r, alors l'equation de Bellman de la value fonction est :

$$V(s) = R(s,a,s') + \gamma V(s')$$

R(s,a,s') est le reward immédiat obtenu quand on execute l'action a sur l état s et qu'on obtient comme nouvel l'etat s'.

Gamma est le discount factor.

V(s') est la valeur du nouvel état .



Reinforcement Learning

Supposons que l'on veuille calcules la valeur de l'état s2.

Appliquons l'equation de Bellman $V(s_2) = R(s_2, a_2, s_3) + \gamma V(s_3)$

$$V(s_2) = r_2 + \gamma V(s_3)$$

D'apres l'episode † précéd $V^{\pi}(s) = R(s, a, s') + \gamma V^{\pi}(s')$
Donc

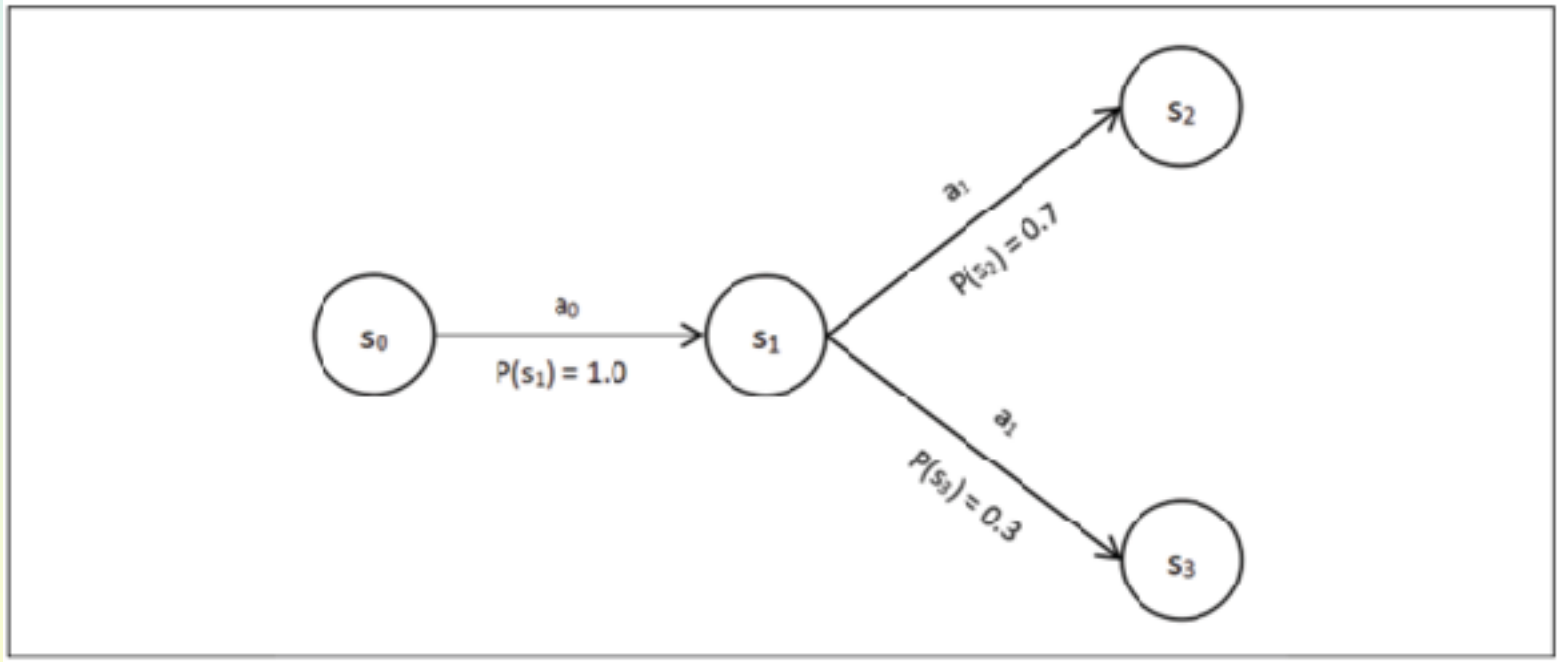
donc de manière générale :

Reinforcement Learning

L'equation de Bellman precedente ne concerne qu'un environnement deterministe.

Supposons que notre environnement soit stochastique.

Si nous effectuons une action a dans un état s:



Reinforcement Learning

Donc en effectuant l'action a1 dans l'état s1, il y a 70% de chances que l'etat suivant soit s2 et 30% s3.

Or l'equation de Bellman représente la somme de la recompense immédiate et d cela valeur actualisée de l'état suivant.

Or: le prochain état n'est pas garanti donc comment définir l'equation de Bellman?

Modifier l'equation de Bellman

$$V^{\pi}(s) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^{\pi}(s')]$$

Donc il faut prendre en compte les attentes (moyenne pondérée par la probabilité de transition correspondante).

Reinforcement Learning

Donc dans l'exemple :

$$V(s_1) = P(s_2|s_1,a_1)[R(s_1,a_1,s_2) + V(s_2)] + P(s_3|s_1,a_1)[R(s_1,a_1,s_3) + V(s_3)]$$

$$V(s_1) = 0.70[R(s_1,a_1,s_2) + V(s_2)] + 0.30[R(s_1,a_1,s_3) + V(s_3)]$$

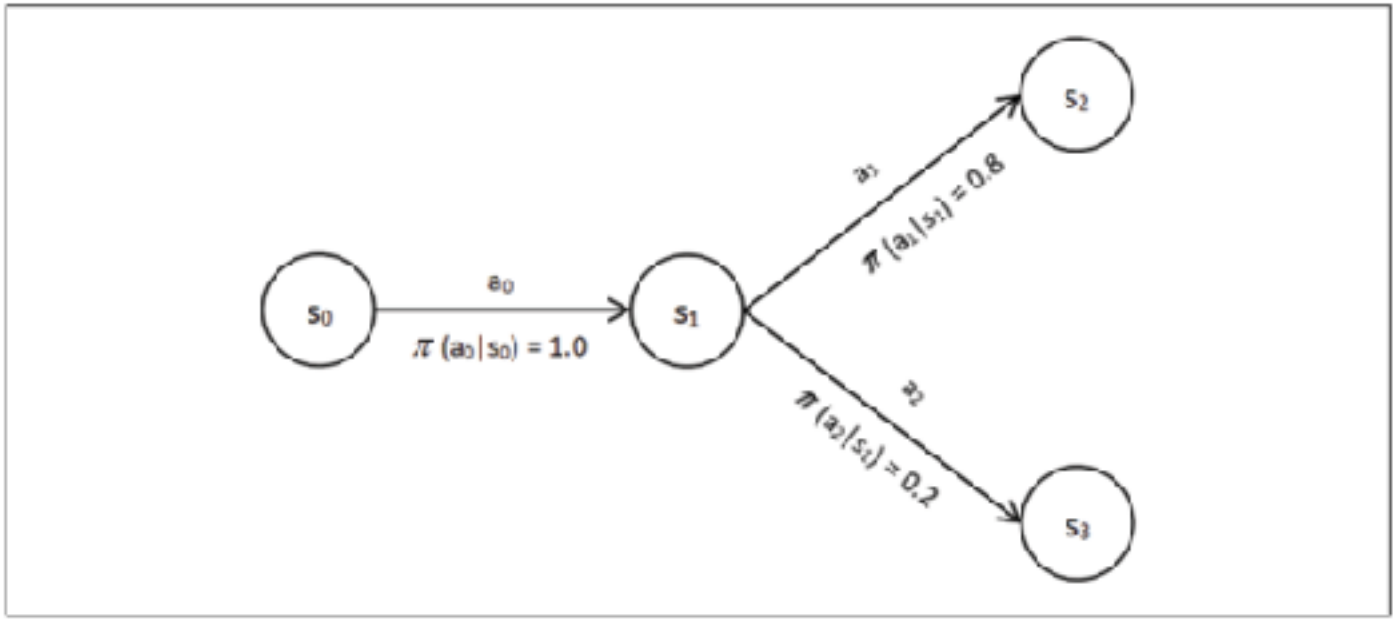
Donc on peut réécrire l'equation de Bellman :

$$V^{\pi}(s) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^{\pi}(s')]$$

Reinforcement Learning

Ok mais que se passe t-il si la politique est stochastique?

Les actions sont sélectionnées à l'aide d'une distribution de probabilités par exemple :



Reinforcement Learning

Pour inclure la nature stochastique de la politique dans l'equation de Bellman, nous pouvons utiliser l'espérance : (moyenne pondérée) - en utilisant la probabilité d'action :

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^{\pi}(s')]$$

$$\mathbb{E}_{x \sim p(x)}[f(X)] = \sum_{x} p(x) f(x)$$

On peut l'exprimer par la Bellman Expectation Equation à l'aide de la formulation: avec

$$f(x) = R(s,a,s') + \gamma V^{\pi}(s') \qquad P(x) = P(s'|s,a) \qquad \pi(a|s)$$

$$V^{\pi}(s) = \mathop{\mathbb{E}}_{\substack{a \sim \pi \\ s' \sim P}} [R(s,a,s') + \gamma V^{\pi}(s')]$$

Reinforcement Learning

Voyons maintenant comment exprimer l'equation de Bellman pour la Q function.

- Similaire à l'equation de Bellman de la fonction valeur précédente.

Rappel : l'equation de Bellman de la Q function va indiquer que la valeur Q d'une paire état-action peut être obtenue comme la somme de la récompense immédiate et de la valeur Q actualisée de la prochaine paire état-action:

$$Q(s,a) = R(s,a,s') + \gamma Q(s',a')$$

En raisonnant come précédent, la Q function peut être exprimée par en considérant une politiqu

$$Q^{\pi}(s,a) = R(s,a,s') + \gamma Q^{\pi}(s',a')$$

Reinforcement Learning

Comme précédemment, cette équation ne fonctionne que dans le cas déterministe.

Si environnement stochastique, il suffit d'utiliser la formule de l'expectation:

$$Q^{\pi}(s,a) = \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma Q^{\pi}(s',a')]$$

De même avec une politique stochastique, réécriture avec l'espérance (poyenne pondérée) en utilisant la probabilité d'action.

$$Q^{\pi}(s,a) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma Q^{\pi}(s',a')]$$

Reinforcement Learning

Attention le terme de probabilité d'action n'est pas nécessaire puisque nous sélectionnons aucune action à partir d'une politique puisque l'action est donnée par Q(s,a).

Par contre nous devons sélectionner une action a' basée sur la politique π tout en calculant la Q value de la prochaine paire état-action Q(s',a') car a' n'est pas donné.

On va donc introduire le terme $\sum_{a'} \pi(a'|s')$ avant Q(s',a')

L'equation de Bellman pour la Q function est donc :

$$Q^{\pi}(s,a) = \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^{\pi}(s',a') \right]$$

Elle peut alors s'exprimer par avec l'esperance :

$$Q^{\pi}(s,a) = \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma \mathbb{E}_{a' \sim \pi} Q^{\pi}(s',a')]$$

Reinforcement Learning

Comment obtenir la valeur optimale?

L'equation d'optimalité de Bellman donne: (1) la valeur optimale de Bellman et (2) valeur optimale des Q functions.

Valeur optimale de Bellman

Rappel Equation de Bellman du calcul des valeurs qui depend de la politique π donc autant que de politiques possibles.

$$V^{\pi}(s) = \mathop{\mathbb{E}}_{\substack{a \sim \pi \\ s' \sim P}} [R(s, a, s') + \gamma V^{\pi}(s')]$$

La fonction de valeur optimale est celle qui donne la valeur maximale par rapport à toutes les autres valeurs de Bellman.

Reinforcement Learning

La fonction de valeur optimale notée V*(s) est celle qui donne la valeur maximale par rapport à toutes les autres valeurs de Bellman.

Comment la calculer?

Le plus simple en sélectionnant l'action qui donne la valeur maximale. Or on ne connait pas cette action.

Il faut donc calculer la valeur de l'état en utilisant toutes les actions possibles et sélectionner la valeur maximale.

On ne va pas utiliser une politique pour sélectionner l'action mais nous allons calculer la valeur de l'état en utilisant toutes les actions possibles puis sélectionner la valeur maximale comme valeur de l'état.

Puisque nous n'utilisons aucune politique, nous pouvons la supprimer et ajouter le maximum sur l'action

Reinforcement Learning

DONC obtention de la valeur Bellman optimale:

$$V^*(s) = \max_a \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma V^*(s')]$$

Meme chose que l'equation de Bellman sauf que nous prenons le maximum sur toutes les actions possibles au lieu de faire la moyenne pondérée sur la politique puisque nous voulons la valeur optimale.

Prenons un exemple : supposons que nous soyons un état s et que nous ayons 2 actions possibles dans cet état (notons les actions 0 et 1).

V*(s) est donnée par

$$V^*(s) = \max \begin{pmatrix} \mathbb{E}_{s' \sim P}[R(s,0,s') + \gamma V^*(s')] \\ \mathbb{E}_{s' \sim P}[R(s,1,s') + \gamma V^*(s')] \end{pmatrix}$$

Reinforcement Learning

Calcul de la valeur optimale des Q fonctions.

Rappel : Equation de Bellman des Q functions.

$$Q^{\pi}(s,a) = \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma \mathbb{E}_{a' \sim \pi} Q^{\pi}(s',a')]$$

Comme précédemment au lieu d'utiliser la politique pour sélectionner une action a' dans l'eat suivant s', nous choisissons toutes les actions possibles dans cet état s' et calculons la valeur Q maximale:

$$Q^{*}(s,a) = \mathbb{E}_{s' \sim P}\left[R(s,a,s') + \gamma \max_{a'} Q^{*}(s',a')\right]$$

Reinforcement Learning

Prenons un exemple : on est dans un état s avec une action a qui nous amène dans l'état s'.

Nous devons calculer la Q value pour l'état suivant s'.

Il peut y avoir plusieurs actions a partir de l'état s4 (supposons 2 actions 0 et 1)

La valeur maximale de la Q function peut s'écrire alors

$$Q^*(s,a) = \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma \max \begin{pmatrix} Q^*(s',0) \\ Q^*(s',1) \end{pmatrix}]$$

Reinforcement Learning

Dans les deux cas (calcul de la valeur de Bellman optimale et valeur de la Q fonction optimale), on a donc:

$$V^*(s) = \max_a \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma V^*(s')]$$

$$Q^*(s,a) = \mathbb{E}_{s' \sim P}\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$

On peut alors developper l'espérance et obtenir:

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')]$$

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')]$$

Reinforcement Learning

Les relations entre les State value (valeur de Bellman V(s)) et les Q value (Q(s,a).

La valeur d'un état (V(s) - value function - représente le rendement attendu à partir de cet état s suivant une politique π

$$V^{\pi}(s) = \mathop{\mathbb{E}}_{\tau \sim \pi} [R(\tau) | s_0 = s]$$

De meme, la Q value d'une paire état-action (Q function) représente le reward attendu à partir de la paire (etat-action-) en suivant une politique π:

$$Q^{\pi}(s, a) = \mathop{\mathbb{E}}_{\tau \sim \pi} [R(\tau) | s_0 = s, a_0 = a]$$

Reinforcement Learning

la valeur optimale par rapport à toutes les politiques est obtenue par le maximum de la state value sur toutes les politiques:

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

D'autre part , la Q function optimale pour une politique est donné par le maximum des Q value état-action:

$$Q^{\pi}(s,a) = \max_{\pi} Q^{\pi}(s,a)$$

Y a t-il une relation entre la fonction de valeur optimale (V*(s)) et la Q function optimale ($Q^{\pi}(s,a)$)?

Puisque la fonction de valeur optimale est le rendement maximum attendu en partant de l'état s et que la Q fonction optimale représente me rendement maximum attendu lorsque nous partons de l'état s en effectuant l'action a, nous avons une relation.

Reinforcement Learning

la fonction de valeur optimale V*(s) est le maximum de la valeur Q optimale sur toutes les actions possibles:

$$V^*(s) = \max_a Q^*(s,a)$$

Relations entre les equations de Bellman

Rappel:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^\pi(s')]$$

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a')]$$

Reinforcement Learning

De plus

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')]$$

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')]$$

Donc

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')]$$

et donc

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')]$$

puisque

$$V^*(s) = \max_a Q^*(s,a)$$

Reinforcement Learning

Programmation Dynamique (Dynamic Programming -DP)

La programmation dynamique (DP)est une technique de resolution de problèmes complexes

En DP, au lieu de résoudre un pb dans son ensemble, on le decompose en sous-problèmes simples (que l'on résout et dont on stocke le résultat). Si le meme sous-problème se produit, on ne le recalcule pas car on utilise la solution deja calculée.

2 méthodes utilisant la DP permettent de trouver la politique optimale :

-Value Iteration

-Policy iteration

Attention : DP est une méthode model-based (on doit connaitre les probabilités de transition de l'environnement.)

Reinforcement Learning

Value Iteration

But Trouver la politique optimale.

La politique optimale est celle qui dit à l'agent d'effectuer l'action correcte dans chaque état.

Pour calculer la politique optimale :

1-calculer la fonction de valeur optimale

2-utiliser le résultat pour déduire la politique optimale.

Partons de l'equation de Bellman optimale de la fonction valeur :

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')]$$

De plus nous avons vu les relations entre la fleur V* et Q*

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')]$$

Reinforcement Learning

En substituant la dernière equation dans la précédente, on obtient

$$V^*(s) = \max_a Q^*(s, a)$$

Nous pouvons donc calculer la fonction de valeur optimale en prenant simplement le maximum sur la Q function optimale.

Donc pour calculer la valeur d'un état, nous calculons la Q value pour toutes les paires état-action. Et ensuite nous sélectionnons la Q value maximale comme valeur de l'état.

Reinforcement Learning

L'algorithme de Value iteration

1- Calculer la fonction de valeur optimale V*(s) en prenant le maximum sur les Q fonctions

$$V^*(s) = \max_a Q^*(s,a)$$

2- Extraire la politique optimale à partir de la value function optimale calculée en 1

Prenons un exemple: Si nous sommes dans l'état A et qu'on doive atteindre l'état C sans passer par l'état B.

Nous avons 2 actions : 0-Left/right et 1 up/dpwn

Reinforcement Learning

Trouvons la politique optimale dans ce cas.

La politique optimale est celle qui dit d'effectuer l'action 1 dans l'état A pour atteindre C sans passer par B. Comment la trouver avec l'iteration de valeur.

Dynamique du model associée à l'état A

State (s)	Action (a)	Next State (s')	Transition Probability $P(s' \mid s,a)$ or $P^a_{ss'}$	Reward Function $R(s,a,s')$ or $R^a_{ss'}$
A	0	A	0.1	0
A	0	B	0.8	-1
A	0	C	0.1	1
A	1	A	0.1	0
A	1	B	0.0	-1
A	1	C	0.9	0

Reinforcement Learning

Etape 1: Calcul de la fonction valeur optimale.

On utilise

$$V^*(s) = \max_a Q^*(s,a)$$

Or on a aussi

$$Q(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V(s')]$$

Pour simplifier on note:

- P(s' I s,a) par $P^a_{ss'}$

Et donc on a

- R(s,a,s') par) par $R^a_{ss'}$

Et donc on obtient:

$$Q(s,a) = \sum_{s'} P^a_{ss'}\left[R^a_{ss'} + \gamma V(s')\right]$$

Reinforcement Learning

Donc en utilisant la formule précédente, on peut calculer la Q function avec :

. la probabilité de transition $P^a_{ss'}$

. la fonction de reward $R^a_{ss'}$

. la valeur de l'état suivant V(s')

La dynamique du modèle donne la probabilité de transition et la fonction Reward.

Mais comment trouver la valeur de l'état suivant puisqu'on ne connait pas encore la valeur des états?

On va les initialiser à des valeurs aléatoires opus à des zéros.

Reinforcement Learning

Iteration 1: Calcul de la Q value de l'etat A.

Nous avons 2 actions 0 et 1

Nous calculons tout d'abord la Q value pour l'état A et l'action 0

Pour tout l'exemple prenons le discount factor gamma = 1

$$
\begin{aligned}
Q(A,0) &= P^0_{AA}[R^0_{AA} + \gamma V(A)] + P^0_{AB}[R^0_{AB} + \gamma V(B)] + P^0_{AC}[R^0_{AC} + \gamma V(C)] \\
&= 0.1(0+0) + 0.8(-1+0) + 0.1(1+0) \\
&= -0.7
\end{aligned}
$$

Reinforcement Learning

De meme calculons Q(A,1)

$$
\begin{aligned}
Q(A,1) &= P^1_{AA}[R^1_{AA} + \gamma V(A)] + P^1_{AB}[R^1_{AB} + \gamma V(B)] + P^1_{AC}[R^1_{AC} + \gamma V(C)] \\
&= 0.1(0+0) + 0.0(-1+0) + 0.9(1+0) \\
&= 0.9
\end{aligned}
$$

Apres ce calcul, nous pouvons mettre à jour la Q table

State	Action	Value
A	0	- 0.7
A	1	0.9
B	0	
B	1	
C	0	
C	1	

Reinforcement Learning

Nous pouvons alors déduire la valeur optimale qui est juste le maximum de la Q function.

Donc V(A) = Q(A,1) = 0.9.

On peut alors mettre à jour la valeur de l'état A dans le tableau des valeurs

State	Value
A	0.9
B	0
C	0

Reinforcement Learning

De meme on peut calculer la valeur de l'état B(V(B)).

On calcule la Q value de Q(B,0) et Q(B,1) et on sélectionne la Q value la plus élevée.

On fait de même pour les valeurs des autres états et on obtient la table des valeurs des états à la fin de l'iteration 1:

State	Value
A	0.9
B	- 0.2
C	0.5

MAIS les valeurs de la table obtenues après l'iteration 1 ne sont pas les valeurs optimales car le calcul est basé sur des valeurs initialisées aléatoirement.

Reinforcement Learning

Iteration 1 : les valeurs des états suivants sont initialisées aléatoirement

$$V^*(s) = \max_a Q^*(s,a)$$

$$\sum_{s'} P^a_{ss'}[R^a_{ss'} + \gamma V(s')]$$

Random Values

Donc nous allons utiliser les valeurs obtenues lors de l'itération précédentes pour les autres itérations suivantes

Donc dans la deuxième itération, pour calculer lales valeurs des états, nous calculons la Q value de toutes les paires etat-action et sélectionnons la valeur maximale en utilisant les valeurs d'état calculée lors de la premiere itération.

Reinforcement Learning

Simplement voila ce que nous faisons:

$$V^*(s) = \max_a \underline{Q^*(s,a)}$$

$$\sum_{s'} P^a_{ss'}[R^a_{ss'} + \gamma \underline{V(s')}]$$

Use state values from 1st iteration

Reinforcement Learning

Itération 2 : on calcule les Q value.

$$
\begin{aligned}
Q(A,0) &= P^0_{AA}[R^0_{AA} + \gamma V(A)] + P^0_{AB}[R^0_{AB} + \gamma V(B)] + P^0_{AC}[R^0_{AC} + \gamma V(C)] \\
&= 0.1(0 + 0.9) + 0.8(-1 - 0.2) + 0.1(1 + 0.5) \\
&= -0.72
\end{aligned}
$$

$$
\begin{aligned}
Q(A,1) &= P^1_{AA}[R^1_{AA} + \gamma V(A)] + P^1_{AB}[R^1_{AB} + \gamma V(B)] + P^1_{AC}[R^1_{AC} + \gamma V(C)] \\
&= 0.1(0 + 0.9) + 0.0(-1 - 0.2) + 0.9(1 + 0.5) \\
&= 1.44
\end{aligned}
$$

D'ou la nouvelle table des valeurs d'état

State	Value
A	1.44
B	- 0.50
C	1.0

Reinforcement Learning

Itération 3 : on obtient de meme. la table suivante:

State	Value
A	1.94
B	- 0.70
C	1.3

Nous continuons ainsi jusqu'à trouver la Value function optimale.

Mais comment savoir quand on a trouver les valeurs maximale?

On le sait quand les valeurs de la table des etats ne changent plus après les iterations (ou changent très peu)

Reinforcement Learning

Par exemple après l'iteration 4 on obtient

State	Value
A	1.94
B	- 0.70
C	1.3

Nous pouvons remarquer que la différence entre la table des valeurs obtenue après l'itération 4 et après l'itération 3 est très faible.

Nous pouvons alors dire que nous avons atteint la convergence et nous prenons la table des valeurs obtenue pares l'itération 4 comme fonction de valeur optimale.

Evidemment en général cela demande de nombreuses iterations.

Reinforcement Learning

Etape 2 : Extraction de la politique optimale (pares obtention des valeurs optimales des états a la fin de l'étape 1)

A la fin de l'étape 1 on a obtenu:

State	Value
A	1.95
B	- 0.72
C	1.3

Pour extraire la politique optimale, il faut sélectionner l'action que correspond au maximum de la Q value dans chaque état.

Reinforcement Learning

Nous avons la formule pour calculer les Q value :

$$Q(s,a) = \sum_{s'} P_{ss'}^{a}[R_{ss'}^{a} + \gamma V(s')]$$

Nous allons utiliser les valeurs optimales des états obtenues a la fin de l'étape 1 pour calculer les Q values.

Ensuite nous allons pouvoir extraire la politique optimale en sélectionnant l'action qui correspond à la Q value maximum puisque

$$\pi^* = \arg\max_{a} Q(s,a)$$

Reinforcement Learning

Donc utilisons les valeurs maximales des états obtenues précédemment pour calculer les Q values

$$
\begin{aligned}
Q(A,0) &= P^0_{AA}[R^0_{AA} + \gamma V(A)] + P^0_{AB}[R^0_{AB} + \gamma V(B)] + P^0_{AC}[R^0_{AC} + \gamma V(C)] \\
&= 0.1(0 + 1.95) + 0.8(-1 - 0.72) + 0.1(1 + 1.3) \\
&= -0.951
\end{aligned}
$$

$$
\begin{aligned}
Q(A,1) &= P^1_{AA}[R^1_{AA} + \gamma V(A)] + P^1_{AB}[R^1_{AB} + \gamma V(B)] + P^1_{AC}[R^1_{AC} + \gamma V(C)] \\
&= 0.1(0 + 1.95) + 0.0(-1 - 0.72) + 0.9(1 + 1.3) \\
&= 2.26
\end{aligned}
$$

Puisque Q'A,1) est supérieure à Q(A,0), l'algorithme de recherche de la politique optimale sélectionnera l'action 1 comme action optimale dans l'état A.

Reinforcement Learning

Voila le résultats obtenu après avoir calculé les Q values pour toutes les paires état-action en utilisant les valeurs des états optimales obtenues après l'étape 1

State	Action	Value
A	0	- 0.95
A	1	2.26
B	0	- 0.5
B	1	0.5
C	0	- 1.1
C	1	1.4

A partir de cette table, nous allons donc sélectionner l'action 1 dans l'état A, l'action 1 dans l'état B et l'action 1 dans l'état C.

Donc la politique optimale obtenue dit que dans l'état A, on choisit l'action 1 qui nous amène à C sans passer par B.

Reinforcement Learning

2eme méthode. : Policy iteration

Nous allons calculer la fonction de valeur optimale en utilisant la politique de manière itérative, une fois que nous avons trouvé la fonction de valeur optimale, nous pouvons l'utiliser pour extraire la politique optimale.

Supposons que nous ayons une politique π, comment l'utiliser pour calculer la fonction de valeur en utilisant π?

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^{\pi}(s')]$$

SI politique déterministe , nous pouvons supprimer le terme $\sum_{a} \pi(a|s)$

On obtient alors

$$V^{\pi}(s) = \sum_{s'} P_{ss'}^{a}\left[R_{ss'}^{a} + \gamma V(s')\right]$$

Reinforcement Learning

Comme précédemment il faut initialiser la politique aléatoirement et calculer la fonction de valeur à l'aide de la politique aléatoire.

Ensuite on vérifie si la fonction de valeur calculée est optimale ou non.

Ensuite on vérifie si la fonction de valeur calculée est optimale ou non.

On va alors extraire une nouvelle politique de la fonction de valeur calculée et nous utilisons la nouvelle politique pour calculer la nouvelle politique etc…

Des que la politique est optimale , l'itération s'arrête.

Reinforcement Learning

Iteration 1:

Soit π_0 la politique aléatoire. On utilise cette politique aléatoire pour calculer la value fonction V^{π_0}

Cette Value fonction V^{π_0} n'est pas optimale car calculée sur la base de la politique aléatoire

A partir de cette Value Fonction V^{π_0} , on va pouvoir calculer une nouvelle politique π_1

Iteration 2:

On utilise π1 la nouvelle politique pour calculer la value fonction V^{π_1}

On vérifie si cette value fonction est optimale. Si oui on a fini; Si non , on extrait la nouvelle politique π_2

Reinforcement Learning

On va répéter le processus jusqu'à obtenir une value fonction optimale

$$\pi_0 \rightarrow V^{\pi_0} \rightarrow \pi_1 \rightarrow V^{\pi_1} \rightarrow \pi_2 \rightarrow V^{\pi_2} \rightarrow \pi_3 \rightarrow V^{\pi_3} \rightarrow \cdots \rightarrow \pi^* \rightarrow V^{\pi^*}$$

Ce processus permet l'evaluation et l'amelioration de la politique.

L'evaluation de la politique implique qu'à chaque itération, nous vérifions si la value fonction calculée à l'aide de cette politique est optimale.

L'amelioration de la politique signifie qu'à chaque itération nous trouvons une nouvelle politique améliorée pour calculer la Value fonction optimale.

Remarque : la politique optimale est celle qui est la meme que la politique obtenue à l'iteration précédente

Reinforcement Learning

Pseudo-code

```
policy = random_policy
for i in range(num_iterations):
    value_function = compute_value_function(policy)
    new_policy = extract_policy(value_function)
    if value_function = optimal:
        break
    else:
        policy = new_policy
```

Reinforcement Learning

Comment savoir si la value fonction est optimale?

Comme précedemment si elle n'a pas changé entre 2 itérations c'est qu'elle est optimale

```
policy = random_policy
for i in range(num_iterations):
    value_function = compute_value_function(policy)
    new_policy = extract_policy(value_function)
    if policy == new_policy:
        break
    else:
        policy = new_policy
```

Reinforcement Learning

Est ce que les méthodes de DP sont applicables à tous les problèmes?

Dans la méthode Value iteration, on utilise

$$V^*(s) = \max_a Q^*(s,a)$$

avec $$Q(s,a) = \sum_{s'} P^a_{ss'}[R^a_{ss'} + \gamma V(s')]$$

Dans la méthode Policy iteration, on utilise

$$V^\pi(s) = \sum_{s'} P^a_{ss'}[R^a_{ss'} + \gamma V^\pi(s')]$$

Reinforcement Learning

Dans les deux cas, il faut calculer la value fonction et la Q fonction.

Mais pour cela , il faut connaitre la probabilité de transition.

Donc la DP implique des méthodes basées sur un modèle et donc il faut connaitre la dynamique du modèle (probabilité de transition).

Si nous ne connaissons pas la dynamique des modèle d l'environnement, il faut utiliser d'autres méthodes , appelées Model-free.

Une de ces méthodes est appelée Monte Carlo